

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	PRUEBAS DE SOFTWARE				
TÍTULO DE LA PRÁCTICA:	Pruebas de Integración: API Testing con Postman y Supertest				
NÚMERO DE PRÁCTICA:	08	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	X	MÁXIMO DE ESTUDIANTES	4	
FECHA INICIO:	25/06/2026	FECHA FIN:	01/07/2026	DURACIÓN:	100 min
RECURSOS A UTILIZAR: - Una computadora personal - Node.js / Supertest - Visual Studio Code - Postman					
DOCENTE(s): Prof. Robert Arisaca / Prof. Diego Iquira / Prof. Lino Pinto					

OBJETIVOS/TEMAS Y COMPETENCIAS
OBJETIVOS: - Validar la comunicación y el flujo de datos entre diferentes módulos o servicios mediante Pruebas de Integración. - Automatizar la verificación de contratos de API y protocolos de comunicación en arquitecturas distribuidas utilizando Postman y Supertest.
TEMAS: - Pruebas de Integración (Small & Large) - API Testing - Automatización con Supertest - Gestión de Colecciones en Postman.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 2

COMPETENCIAS	C.m. Construye responsablemente soluciones siguiendo un proceso adecuado llevando a cabo las pruebas ajustadas a los recursos disponibles del cliente
---------------------	---

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

1. Pruebas de Integración

Las pruebas de integración constituyen el segundo nivel fundamental del Modelo en V, ejecutándose una vez que las unidades de software individuales han sido validadas. Su objetivo principal no es verificar la lógica interna de un módulo, sino exponer defectos en las interfaces y en la interacción entre componentes integrados. Mientras que las pruebas unitarias aíslan el código, la integración busca detectar problemas de interoperabilidad, desajustes de datos y fallos en el flujo de trabajo entre servicios.

Existen dos alcances definidos para este nivel:

- **Integración "en pequeña" (Small):** Se enfoca en las interfaces entre componentes internos o subsistemas del propio software.
- **Integración "en grande" (Large):** También llamada integración de sistemas, verifica la comunicación con servicios externos, bases de datos o productos **COTS** (Commercial Off-the-Shelf).

2. El Dominio de las Pruebas de API (API Testing)

En el ecosistema tecnológico actual, dominado por arquitecturas de microservicios y computación *serverless*, las pruebas de API han superado en importancia a las de interfaz de usuario (UI).

- **Estabilidad y Velocidad:** Las pruebas de API son significativamente más rápidas de ejecutar y más estables que las de UI, ya que no dependen de cambios cosméticos en el diseño.
- **Validación de Contratos:** El software moderno se construye como módulos débilmente acoplados que dependen de **contratos explícitos**. Las pruebas de integración aseguran que cada componente cumpla con su parte del contrato, evitando regresiones silenciosas.
- **Seguridad y Robustez:** Permiten validar protocolos de comunicación (REST/GraphQL) y el manejo de errores ante datos malformados antes de que lleguen a la capa de presentación.

3. Estrategias de Integración e Infraestructura Técnica

Para evitar el enfoque de **"Big Bang"** (unir todo al final), que dificulta enormemente la depuración de fallos, se utilizan estrategias incrementales:

- **Top-Down (De arriba hacia abajo):** Comienza con los módulos superiores. Requiere el uso de Stubs, que son simuladores rudimentarios de componentes inferiores aún no desarrollados.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 3

- **Bottom-Up (De abajo hacia arriba):** Comienza con los componentes básicos. Utiliza Drivers, programas controladores que simulan las llamadas de módulos superiores.
- **Mocks y Dummies:** A diferencia de los stubs, los Mocks u objetos simulados ofrecen una funcionalidad más avanzada para disparar comportamientos específicos durante la prueba.

4. Herramientas Actuales: Postman y Supertest

La elección de herramientas impacta directamente en la capacidad de escalar la calidad:

- **Postman:** Es la plataforma líder para el diseño y prueba de APIs. Permite la colaboración mediante **Colecciones** compartidas y la gestión de múltiples entornos (Desarrollo, Staging, Producción) mediante variables. Su integración en tuberías de CI/CD se realiza a través de Newman, asegurando que las pruebas corran automáticamente con cada *commit*.
- **Supertest:** Es una biblioteca de Node.js que proporciona una abstracción de alto nivel para probar servidores HTTP. Es ideal para equipos que prefieren pruebas basadas en código que puedan versionarse en el mismo repositorio de la aplicación. Permite realizar aserciones complejas sobre el cuerpo de la respuesta, cabeceras y estados HTTP de forma programática.

5. Calidad Moderna: Shift-Left y Uso de la IA

La ingeniería de software actual exige un desplazamiento a la izquierda (Shift-Left), integrando la validación desde la definición de los requisitos.

- **Observabilidad y Retroalimentación:** Los defectos detectados en producción se alimentan de vuelta a las tuberías automatizadas, creando bucles de calidad continua.
- **IA Generativa en el Testing:** Para 2026, la IA automatiza la generación de datos sintéticos realistas para evitar el uso de datos productivos sensibles. Herramientas avanzadas permiten ahora la **autocuración (Self-healing)** de scripts de prueba, donde el sistema detecta cambios en las estructuras de los datos y ajusta los casos de prueba automáticamente, reduciendo el mantenimiento manual en un 60-70%.
- **Análisis Predictivo:** Se utilizan algoritmos para identificar qué módulos tienen mayor probabilidad de fallo basándose en el historial de cambios, priorizando así los esfuerzos de integración en las áreas de mayor riesgo.

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

Ejercicio 1: Pruebas de Integración en Postman (Flujo CRUD)

Para este ejercicio, construiremos un Flujo CRUD completo de Productos. Primero montaremos un servidor rápido en Node.js + Express y luego configuraremos la estrategia de pruebas encadenadas en Postman.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 4

Parte 1: El Servidor Backend

Para que Postman tenga una API real a la cual trabajar, crearemos un servidor local que gestione productos en memoria.

Paso 1: Configuración del proyecto Node.js

1. Crea una carpeta vacía en tu computadora.
2. Abre una terminal dentro de esa carpeta y ejecuta:

```
npm init -y
```

```
npm install express
```
3. Creamos un archivo server.js y utilizamos el archivo compartido
4. Para levantar el servidor ejecutamos el comando

```
node server.js
```

Parte 2: Configuración en Postman

El objetivo de la prueba de integración es validar que el ciclo de vida del producto funcione de inicio a fin utilizando variables de entorno para pasar el ID del producto entre peticiones.

Paso 1: Crear el Entorno (Environment)

1. En Postman, ve a la barra lateral izquierda y haz clic en **Environments** -> **Create Environment**.
2. Nómbralo como Desarrollo Local.
3. Añade una variable llamada productid.
4. Hagan clic en Save y asegúrense de seleccionar este entorno en la esquina superior derecha de Postman.

Paso 2: El Flujo de Peticiones (CRUD)

Añadiremos 4 peticiones consecutivas dentro de una nueva colección en Postman:

1. POST - Crear Producto

- **Método:** POST
- **URL:** http://localhost:3000/api/productos
- **Body (JSON):**

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 5

```
{
  "nombre": "Teclado Mecánico RGB",
  "precio": 85.50
}
```

Pestaña Tests: Aquí capturamos el ID dinámico que nos da Node.js y lo guardamos en nuestro entorno.

```
pm.test("Status 201 - Producto Creado", function () {
  pm.response.to.have.status(201);
});

const res = pm.response.json();
pm.test("Trae un ID válido", function () {
  pm.expect(res).to.have.property("id");
});

// Guardar ID para el siguiente paso del flujo de integración
pm.environment.set("productoId", res.id);
```

2. GET - Leer Producto Creado

- **Método:** GET
- **URL:** `http://localhost:3000/api/productos/{{productId}}`
- **Pestaña Tests:** Validamos que el servidor nos devuelva exactamente el producto que acabamos de registrar.

```
pm.test("Status 200 - Producto Encontrado", function () {
  pm.response.to.have.status(200);
});

const res = pm.response.json();
pm.test("Los datos coinciden con la persistencia", function () {
  pm.expect(res.id).to.eql(pm.environment.get("productoId"));
  pm.expect(res.nombre).to.eql("Teclado Mecánico RGB");
});
```

3. PUT - Actualizar Producto

- **Método:** PUT
- **URL:** `http://localhost:3000/api/productos/{{productId}}`
- **Body (JSON):**

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 6

```
{
  "nombre": "Teclado Mecánico RGB Pro",
  "precio": 99.99
}
```

Pestaña Tests: Verificamos que el cambio de estado se haya procesado de forma correcta en el servidor.

```
pm.test("Status 200 - Actualización Exitosa", function () {
  pm.response.to.have.status(200);
});

const res = pm.response.json();
pm.test("Campos actualizados correctamente", function () {
  pm.expect(res.nombre).to.eql("Teclado Mecánico RGB Pro");
  pm.expect(res.precio).to.eql(99.99);
});
```

4. DELETE - Eliminar Producto

- **Método:** DELETE
- **URL:** http://localhost:3000/api/productos/{{productid}}
- **Pestaña Tests:** Cerramos el ciclo eliminando el recurso y limpiando nuestro entorno de pruebas.

```
pm.test("Status 200 - Producto Eliminado", function () {
  pm.response.to.have.status(200);
});

// Limpieza académica del entorno
pm.environment.unset("productId");
```

Ejercicio 2: Pruebas de Integración con Supertest

Utilizando un entorno de pruebas con Jest y Supertest en Node.js, realiza una prueba de integración para una aplicación Express que simule la gestión de un carrito de compra, se va a verificar

- Añadir un producto al carrito.
- Verificar que el carrito ya no esté vacío y contenga ese producto.

1. El Servidor Express (app.js)

Crea un archivo llamado app.js. Este código configura la API y guarda el carrito en la memoria del servidor.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 7

```
const express = require('express');
const app = express();
app.use(express.json()); // Permitir que la API entienda JSON

// Nuestro carrito empieza vacío
let carrito = [];

// Endpoint 1: Añadir artículo al carrito
app.post('/carrito', (req, res) => {
  const { articulo } = req.body;
  carrito.push(articulo);
  res.status(201).json({ mensaje: "Artículo añadido", carrito });
});

// Endpoint 2: Ver el contenido del carrito
app.get('/carrito', (req, res) => {
  res.status(200).json({ totalArticulos: carrito.length, items: carrito });
});

// Exportamos la app para que Supertest la pueda usar sin encender puertos
module.exports = app;
```

2. La Prueba de Integración (carrito.test.js)

Creamos un archivo llamado carrito.test.js. Aquí es donde Supertest actúa simulando los clics de un usuario real.

```
const request = require('supertest');
const app = require('./app'); // Importamos nuestra API

describe('Prueba de Integración del Carrito de Compras', () => {
  it('Debería añadir un producto y luego mostrarlo en el carrito', async () => {
    // PASO 1: Agregamos un artículo mediante un POST
    const respuestaPost = await request(app)
      .post('/carrito')
      .send({ articulo: 'Monitor Gamer 24" });

    // Validamos que el servidor responda que se creó con éxito (201)
    expect(respuestaPost.statusCode).toBe(201);
    expect(respuestaPost.body.mensaje).toBe("Artículo añadido");

    // -----
    // PASO 2 (INTEGRACIÓN): Consultamos el carrito con un GET
    // para verificar que el estado realmente cambió y el producto sigue ahí.
    const respuestaGet = await request(app)
      .get('/carrito');

    // Validamos que la base de datos temporal guardó el artículo del paso anterior
    expect(respuestaGet.statusCode).toBe(200);
    expect(respuestaGet.body.totalArticulos).toBe(1);
    expect(respuestaGet.body.items).toContain('Monitor Gamer 24"');
  });
});
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 8

III. EJERCICIOS/PROBLEMAS PROPUESTOS

1. Automatización con Supertest

Elijan una temática de su preferencia (ej. catálogo de videojuegos, gestión de una biblioteca musical, reserva de películas, etc.) e implementen una API REST básica en Node.js + Express.

Posteriormente, diseñen y ejecuten una suite completa de pruebas de integración utilizando Supertest y Jest/Mocha.

Requerimientos de la Prueba:

- **Flujo de Persistencia Cruzada:** Validar la integración secuencial entre la creación de un recurso (POST /recurso) y su posterior lectura (GET /recurso/:id), asegurando que el ID generado dinámicamente en el primer endpoint sea el puente de entrada para el segundo.
- **Simulación de Modificación de Estado:** Implementar y verificar un endpoint que altere una propiedad cuantitativa del recurso (ej. restar stock, cambiar nivel, actualizar reproducciones) y confirmar mediante un GET posterior que el cambio se consolidó en el mock de persistencia o base de datos de pruebas.
- **Validación de Robustez (Edge Cases):** Asegurar que el sistema maneje correctamente los desajustes de datos o tipos inválidos (ej. enviar un texto en un campo numérico o dejar campos obligatorios vacíos) devolviendo exactamente el código de estado HTTP 400 (Bad Request) junto con su respectivo mensaje de error.

Entregables: * Archivo de la API (app.js).

- Archivo de pruebas (tema_libre.test.js).
- Captura de pantalla o reporte en texto del resultado de la ejecución en la terminal usando Jest/Mocha.

2. Pruebas de Integración del Proyecto Final

El grupo debe aplicar pruebas de integración sobre la arquitectura de su proyecto de fin de curso. El enfoque debe centrarse en verificar la cohesión y el flujo de datos entre los componentes que han sido desarrollados, El grupo puede elegir la herramienta (Requests, Supertest, REST Assured, Playwright, etc.) que mejor se adapte a su lenguaje de programación.

- **Tarea de Análisis de Errores:**
 1. **Mapeo de la Frontera:** Identifique el punto donde el Subsistema A entrega el control o los datos al Subsistema B.
 2. **Inyección de Fallas de Interfaz:** Diseñe al menos 3 casos de prueba orientados a "romper" la comunicación:
 - **Caso 1 (Sintáctico):** Envíe un objeto con campos faltantes o tipos de datos erróneos.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 9

- **Caso 2 (Semántico):** Envíe valores legales pero fuera de lógica para el receptor (ej. una fecha de entrega anterior a la de pedido).
 - **Caso 3 (Resiliencia):** Simule una latencia alta en la respuesta del subsistema B y analice si el subsistema A maneja el *timeout* o colapsa.
3. **Documentación de Discrepancias:** Por cada falla encontrada, complete un **Reporte de Incidente** que detalle el "Resultado Esperado" vs. el "Resultado Real" observado en la interfaz.

I. CUESTIONARIO

1. Por qué las pruebas unitarias exitosas no garantizan que la integración será exitosa?.
2. Explique la diferencia entre un Stub y un Driver y proporcione un ejemplo de cuándo usó uno de ellos en su proyecto.
3. Según Myers, ¿por qué es arriesgado que el mismo desarrollador que escribió el código de los módulos diseñe también las pruebas de integración?
4. Defina "Integración Incremental" y explique por qué el enfoque "Big Bang" debe evitarse en proyectos complejos
5. ¿Cómo ayuda la herramienta seleccionada por su grupo a detectar "defectos enmascarados" entre sus subsistemas?.

II. REFERENCIAS Y BIBLIOGRAFÍA RECOMENDADAS:

[1] Spillner, A., Software Testing Foundations, 5th Ed., 2021.

[2] Myers, G., The Art of Software Testing, 3rd Ed., 2012.

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:

*Ejercicios propuestos /
Cuestionario*

INSTRUMENTOS:

Rúbrica

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 10

CRITERIOS DE EVALUACIÓN

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (55%)	Insuficiente (30%)	No presenta (0%)
Claridad y completitud de la explicación de los ejercicios propuestos. (30 %)	Todos los ejercicios propuestos han sido explicados de forma muy clara y con la profundidad adecuada.	Los ejercicios propuestos han sido resueltos y explicados, pero de manera muy superficial o general.	Los ejercicios propuestos han sido resueltos, sin embargo, algunos de ellos no han sido explicados.	Los ejercicios han sido resueltos parcialmente y no tienen explicaciones de lo realizado.	No presentan los ejercicios resueltos, o la versión final del informe omite por completo esta sección.
Ejercicios Propuestos (Código fuente) (20 %)	Todos los ejercicios propuestos se ejecutan correctamente y devuelven los resultados esperados.	Los ejercicios propuestos han sido implementados pero con los resultados esperados parciales.	Por lo menos el 2/3 de los ejercicios propuestos han sido implementados con resultados	Por lo menos el 1/3 de los ejercicios propuestos han sido implementados con resultados	No se han implementado los ejercicios propuestos.
Cuestionario (30%)	Todas las respuestas son completas, correctas y proporcionan explicaciones claras, incluyen citas.	La mayoría de las respuestas son correctas, pero algunas carecen de la profundidad esperada, pocas citas.	Algunas respuestas son correctas, pero incompletas o con falta de claridad en las explicaciones, citas no relevantes.	La mayoría de las respuestas son escuetas y/o incompletas, no hay evidencia de haber buscado información que las respalden, sin citas.	No presenta el cuestionario.
Presentación del Informe (20%)	El informe está bien estructurado, con excelente redacción, sin errores	El informe está bien estructurado, con algunos errores menores de redacción	El informe presenta problemas en la organización o redacción, con varios errores	El informe está desorganizado, con múltiples errores de redacción y formato	No presenta el informe.